ANALIZADOR SINTÁCTICO

MANUAL TÉCNICO
ANALIZADOR SINTÁCTICO

Introducción 1
Presentación 2
¿Qué incluye este manual? 2
¿Por qué es importante este manual? 2
Resumen 2
Objetivo 2
Aspectos Técnicos 3
- Herramientas Utilizadas Para el Desarrollo del Programa: 3
· Java 3
· Terminal 3
· NetBeans 3
· Sistema Operativo 3
· JFlex 3
- Requisitos Mínimos del Sistema 3
Documentación 4
- Documentación Interna 4
· Comentarios en Código 4
* Comentarios de clases: 4
* Comentarios de métodos: 5
* Comentarios de variables: 5
* Estructura de una clase de backend: 5
* JFLEX. 6
Para el uso de JFLEX se implementó usando la documentación oficial del sitio web https://www.jflex.de/docu.html. 6
* Algoritmos. 6
- Documentación Externa 7
· Descripción de clases: 7
Frontend 8
Backend 8
Controller 8
· Fase de análisis: 9
· Fase de diseño: 9
· Repositorio en GitHub: 11
· Documentación en JavaDoc: 12
Problemas del Programa 13

Introducción

Este manual técnico proporciona una guía detallada para comprender la lógica detrás del programa “Analizador Sintáctico”, utilizando la interfaz gráfica de java swing y el lenguaje de programación Java, el controlador de versiones Git, github y la implementación de autómatas (através de JFlex) para un fácil moldeado para cualquier lenguaje en futuras versiones. Este manual técnico proporciona una descripción exhaustiva de las características del programa, su funcionamiento interno y los requisitos de instalación y configuración. Además, se incluyen detalles sobre la responsabilidad del desarrollador y la compañía desarrolladora, en cuanto a posibles riesgos asociados con el uso durante sesiones prolongadas o usos inadecuados, cambios en el código o el implemento de mejoras o cambio de sus funciones originales.

Al momento de modificar un carácter o algo diferente en el código fuente sin tomar en cuenta la documentación proporcionada el programa podría dejar de funcionar debidamente, por favor tómelo en cuenta al momento de realizar un cambio o implementar una mejora, por ello se adjunta toda la documentación de análisis y diseño necesaria para comprender el programa.

Presentación

Bienvenidos desarrolladores al manual técnico del Analizador Sintáctico, en este manual podrán encontrar lo necesario para poder entender la lógica y el proceso para poder realizar ajustes, cambios o posibles mejoras al programa.

¿Qué incluye este manual?

En este manual, encontrará una descripción detallada de las características del desarrollo del Analizador Sintáctico, su funcionamiento interno y los requisitos necesarios para implementar o cambiar algo del Analizador Sintáctico desde tu sistema.
- Aspectos destacados:
    - Herramientas utilizadas para el desarrollo del programa
    - Desarrollo del programa
    - Documentación Interna
    - Documentación externa

¿Por qué es importante este manual?

Este manual técnico es esencial para comprender completamente el funcionamiento del Analizador Sintáctico, así como para garantizar una experiencia de desarrollo sencilla, clara y concisa.

Resumen

Este manual técnico facilitará la implementación de una mejora o el cambio de una funcionalidad para cualquier desarrollador, es decir el mantenimiento del programa.

Objetivo

El objetivo de este manual es facilitar la comprensión de la estructura, diseño e implementación del programa, este programa nació con la finalidad de ser escalable, por ello se implementaron buenas prácticas de programación, y se espera que este manual sea fácil de entender e interpretar por medio de las herramientas utilizadas.

Aspectos Técnicos

- Herramientas Utilizadas Para el Desarrollo del Programa:

· Java

Analizador Sintáctico es un programa desarrollado y creado exclusivamente con java. La versión usada de Java para desarrollar el Analizador Sintáctico es la versión “25” y la versión de JSE es la misma “25” y esta misma corresponde a la versión del JDK descargadas desde los sitios oficiales de java existentes en los paquetes sudo de las distribuciones de debian y ubuntu de linux.

· *Terminal*

Para observar los cambios realizados y las pruebas realizadas fue necesario ejecutar el archivo ejecutable .jar desde la terminal de ubuntu 24 y debian 13.

· NetBeans

NetBeans fue el IDE en el cual se realizó el programa, donde se extrajo el archivo .jar,
en el proceso de la creación y gestión del proyecto se utilizó el gestor de proyectos “Maven” y “Java Application”, la versión de NetBeans es la versión “Apache NetBeans IDE 27” y todas las dependencias utilizadas por este yacen descritas en el pom.xml

· Sistema Operativo

El programa Java y Netbeans fue utilizado en dos computadoras distintas, una con ubuntu 24 y otra con debian 13.

· JFlex

Para el analizador sintáctico y léxico se utilizó la versión de JFlex 1.9.1 Configurando el archivo .flex de la manera descrita posteriormente.

- Requisitos Mínimos del Sistema

El programa está diseñado para que cualquier equipo que cuente con una consola (terminal “cmd”) pueda utilizar el programa, debido a que no es un programa que exige demasiados recursos, así que no hay requisitos mínimos (del hardware) propios del sistema para poder ejecutar el programa como tal, pero el programa podría llegarse a ver afectado por: La terminal del usuario (código cerrado o abierto con restricciones), una mala instalación de la JVM o una

exageración (datos muy grandes) en los datos ingresados por el usuario en las entradas que requiere el programa para funcionar.

Documentación

Como motivo de facilitar el trabajo en equipo y realizar programación abierta para todo el público se implementaron distintos métodos de documentación, estas las clasificamos en:

- Documentación Interna

Definiendo la documentación interna como la herramienta en programación que está destinada a ser utilizada por los desarrolladores y miembros del equipo de desarrollo de software durante el proceso de creación de un programa o sistema. Esto porque esta documentación no está dirigida al usuario final, sino que está diseñada para facilitar el entendimiento y la colaboración entre los miembros del equipo de desarrollo.

· Comentarios en Código

Para explicar el código y tener buenas prácticas de programación, en cada clase se comentaron y enunciaron los métodos utilizados. Esto por si en algún momento se desee realizar un cambio en especifico. El formato de la explicación de cada método es utilizando javadoc, donde se explica lo que realiza el método, qué necesita y que retorna en dado caso lo haga.

* Comentarios de clases:

Se describe de forma breve la razón de existir de la clase, así como su autor, la versión de java que se utilizó para hacerla o las clases que pueden estar relacionadas a está y la fecha en la cual se creó y se terminó.

```java
/**
 * The Parser class is the class responsible for be the parser that the program
 * will use then of use lexer.java, (this syntactic analyzer analyze lines
 * whitout lexical erros)
 *
 * @see Token
 * @see TokenType
 * @author YmCris
 * @since Oct 31, 2025
 */
public class Parser {
```

* Comentarios de métodos:

Se describe lo más detalladamente posible la acción que tiene el método, que representan los parámetros y que retorna (si aplica).

```
/**
 * Method responsible for analyzing a <strong>line of text</strong>
 * and adding the words in the line to an array of words according to
 * whether they are tokens or errors.
 *
 * @param line line to be analyzed
 * @param row row in which it is found
 * @throws IOException
 */
public void analyze(String line, int row) throws IOException {
```

* Comentarios de variables:

En el comentario de variables no se utiliza el formato de javadoc porque representa un crecimiento del tamaño de una clase de manera exponencial, por lo tanto solo se hace referencia el tipo de variables que son, ya sean de referencia o primitivas y en algunas muy importantes se hace un comentario simple.

```
// REFERENCE VARIABLES -------------------------------------------------------
private String color;
private String lexeme;

// PRIMITIVE VARIABLES -------------------------------------------------------
private int row;
private int column;
private boolean token;
private boolean comment;
private boolean highlighted;
```

* Estructura de una clase de backend:

En la creación de una clase de frontend se divide el contenido de la clase en base a la siguiente “Plantilla”, separando así todos los elementos más importantes de la clase, para que sea más fácil de entender a todos los programadores, aunque claro que la estructura de esta planilla se ve modificada en muchas clases con métodos sobreescritos, funciones, etc.

```java
/**
 * The HolaMundo class is the class responsible for
 *
 * @author YmCris
 * @since Nov 2, 2025
 */
public class HolaMundo {

    // REFERENCE VARIABLES ---------------------------------------------------

    // PRIMITIVE VARIABLES ---------------------------------------------------

    // INSTANCES -------------------------------------------------------------

    // CONSTRUCTOR METHOD ----------------------------------------------------

    // SPECIFIC METHODS ------------------------------------------------------
    // FUNCTIONS -------------------------------------------------------------

    // GETTERS ---------------------------------------------------------------

    // SETTERS ---------------------------------------------------------------

}
```

* JFLEX.

Para el uso de JFLEX se implementó usando la documentación oficial del sitio web **https://www.jflex.de/docu.html**.

* Algoritmos.

Para resolver problemas computacionales es imprescindible aplicar una lógica secuencial, dividiendo el problema en problemas menores, por ello en los métodos que requieran un gran análisis se implementa la estructura de descripción de algoritmos que consiste en una línea antes de la acción realizar un comentario simple con el número del paso del algoritmo y su descripción.

```java
public void analyze(String line, int row) throws IOException {

    //0. Create the line list to save the words analyzed
    ArrayList<Word> lineWords = new ArrayList<>(initialCapacity: 15);

    //1. Create the Lexer instance with the line
    Lexer lexer = new Lexer(new StringReader(s: line));

    //2. Do the lexer analysis
    lexer.yylex();

    List<Word> words = lexer.getWords();

    //3. Add the lexer words to the line list
    for (Word word : words) {

        //4. Set the row word
        word.setRow(row);

        //5. Save the words to the line words
        lineWords.add(e: word);

    }

    //6. Add the break line
    lineWords.add(new Word(lexeme:"\n", row:0, column:0, token: false));
    Parser parser = new Parser(words);
    parser.parse();

    //7. Save the line words to the output lines list
    outputLines.add(e: lineWords);

}
```

- Documentación Externa

Definiendo la documentación externa como la herramienta en programación que transmite información importante destinada a ser utilizada por usuarios finales, clientes, administradores de sistemas u otras partes interesadas que no forman parte del equipo de desarrollo de software. Esta documentación está diseñada para proporcionar orientación, instrucciones y recursos para el uso adecuado y efectivo del software desarrollado.

· Descripción de clases:

Definiendo una **clase** como una planilla la cual sirve como molde para la creación de instancias o para realizar acciones (Basado en la SRP “Single Responsibility Principle”), comunicándose entre otras clases a través de acciones (Métodos), sin embargo ahora se implementó el concepto de Vista - Modelo - Controlador (MVC), esto para tener mejor estructurado todo el código, separar intereses y responsabilidades para facilitar el desarrollo y su mantenimiento, por lo que se describe de esta manera.

Frontend

- **JDReports** – The JDialog JDReports is the JDialog responsible for displaying the reports

- **JDSaveFile** – The JDialog JDSaveFile is the JDialog responsible for displaying the dialog to save the file

- **JDUploadFile** – The JDialog JDUploadFile is the JDialog responsible for displaying the file chooser

- **JFAnalyzer** – The JFrame JFAnalyzer is the JFrame responsible for displaying the analyzer

Backend

- **Analyzer** – The Analyzer class is the class responsible for do the respective analyzer features
- **ComponentsUpdater** – The ComponentsUpdater class is the class responsible for
- **ErrorType** – The enum ErrorType is the enum responsible for representing the error types that exist in the program
- **ErrorWord** – The ErrorWord class is the class responsible for be an error
- **File** – The File class is the class responsible for have the file's tools to use in the text file input and save file
- **Language** – The Language class is the class responsible for contain the language's elements
- **Lexer** – The Lexer class is the class responsible for
- **Parser** – The Parser class is the class responsible for be the parser that the program will use after using Lexer.java, (this syntactic analyzer analyzes lines without lexical errors)
- **Parser** – The Parser class is the class responsible for start the application showing the initial JFrame 'JFAnalyzer'
- **Token** – The Token class is the class responsible for representing a token
- **TokenType** – The enum TokenType is the enum responsible for representing the token's type
- **Word** – The Word class is the class responsible for be the word

Controller

- **AnalyzerController** – The AnalyzerController class is the class responsible for be the lexical analyzer controller, being the responsible to check the entries, and send information with distinct models
- **FileController** – The FileController class is the class responsible for be the controller of upload file

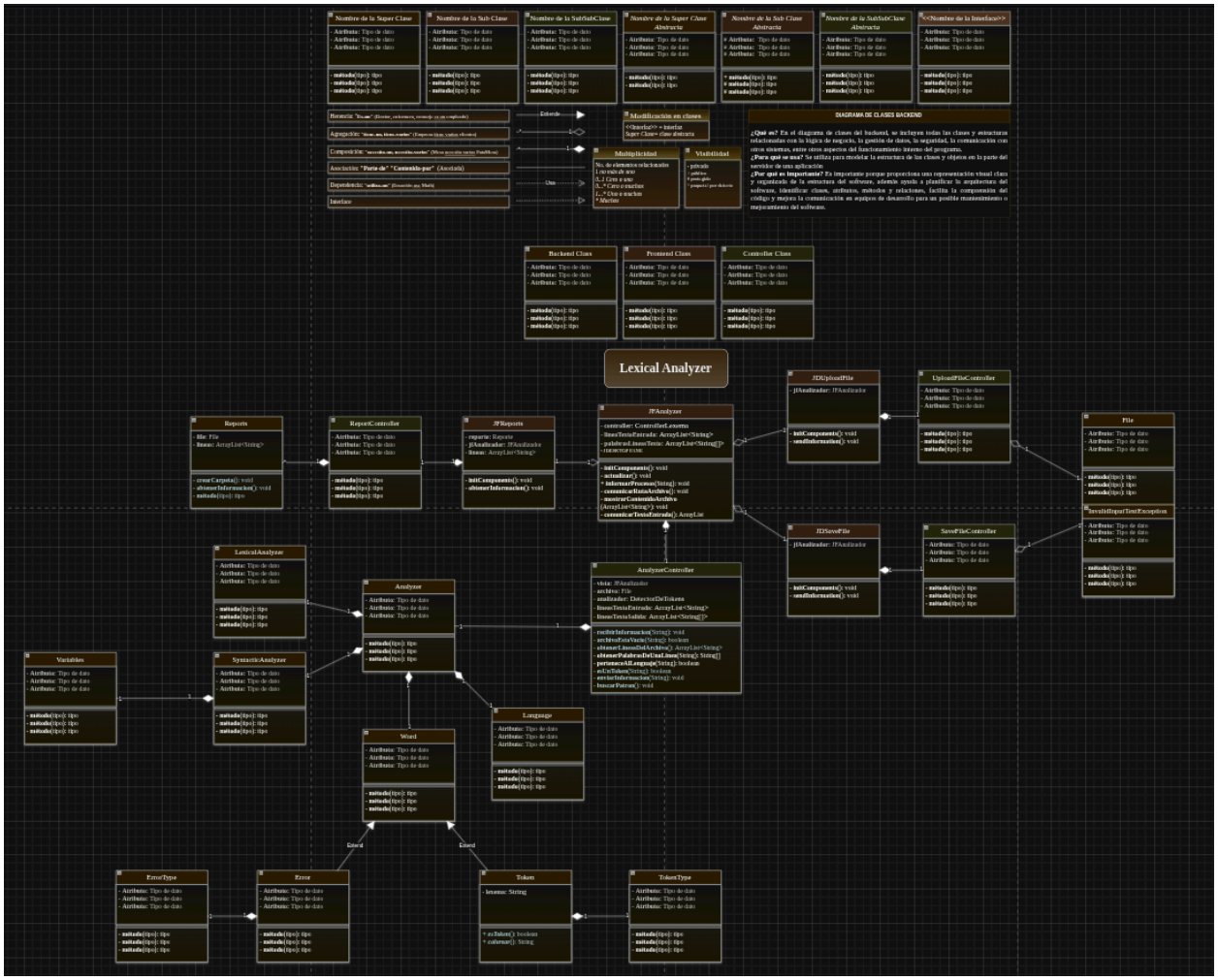
- **ReportsController** – The ReportsController class is the class responsible for be the controller's reports

· Fase de análisis:

Para representar gráficamente el programa se emplea el diagrama de UML diseñado en flowChart.

Fase de Análisis:

1. Diagrama de clases

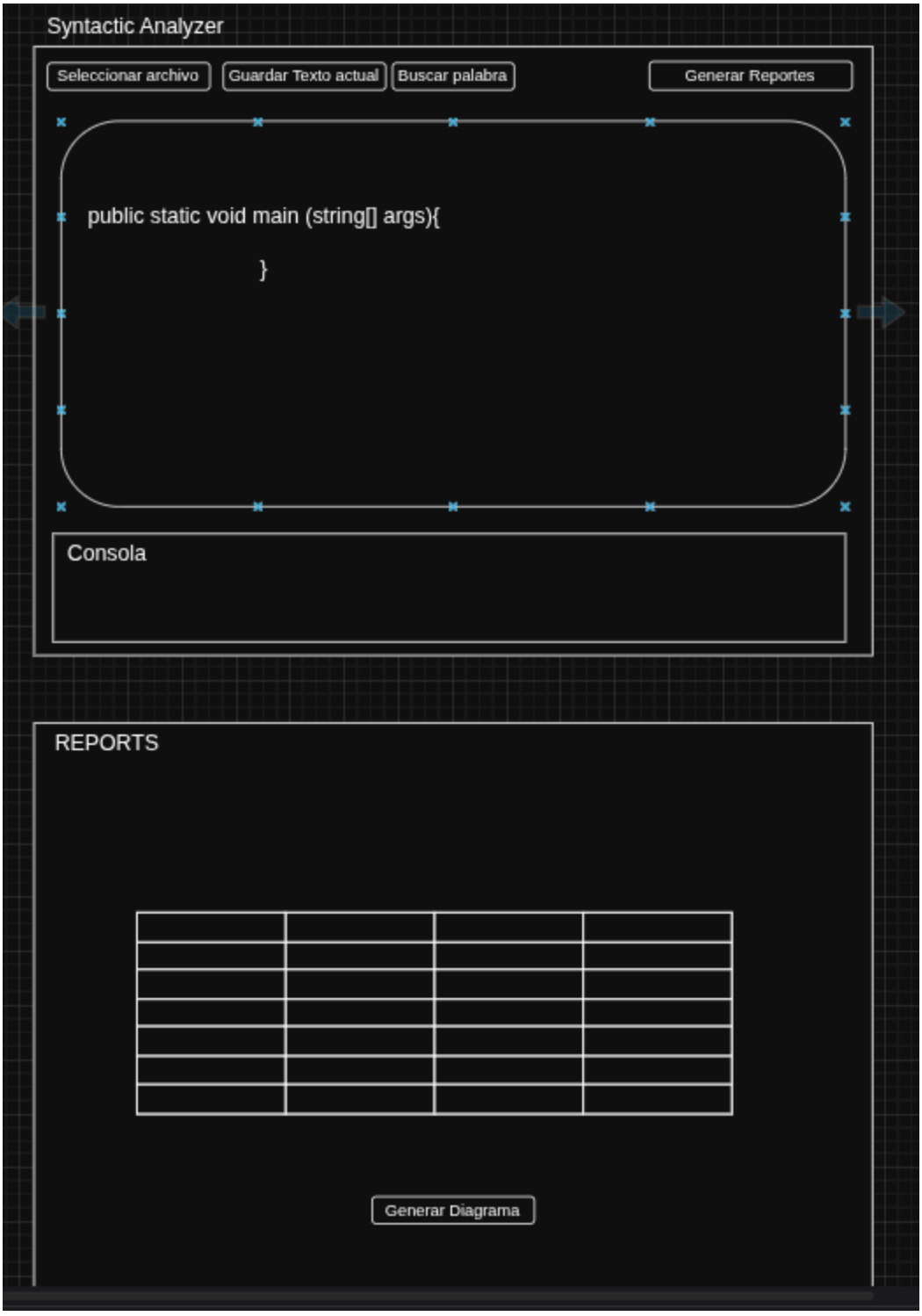


· Fase de diseño:

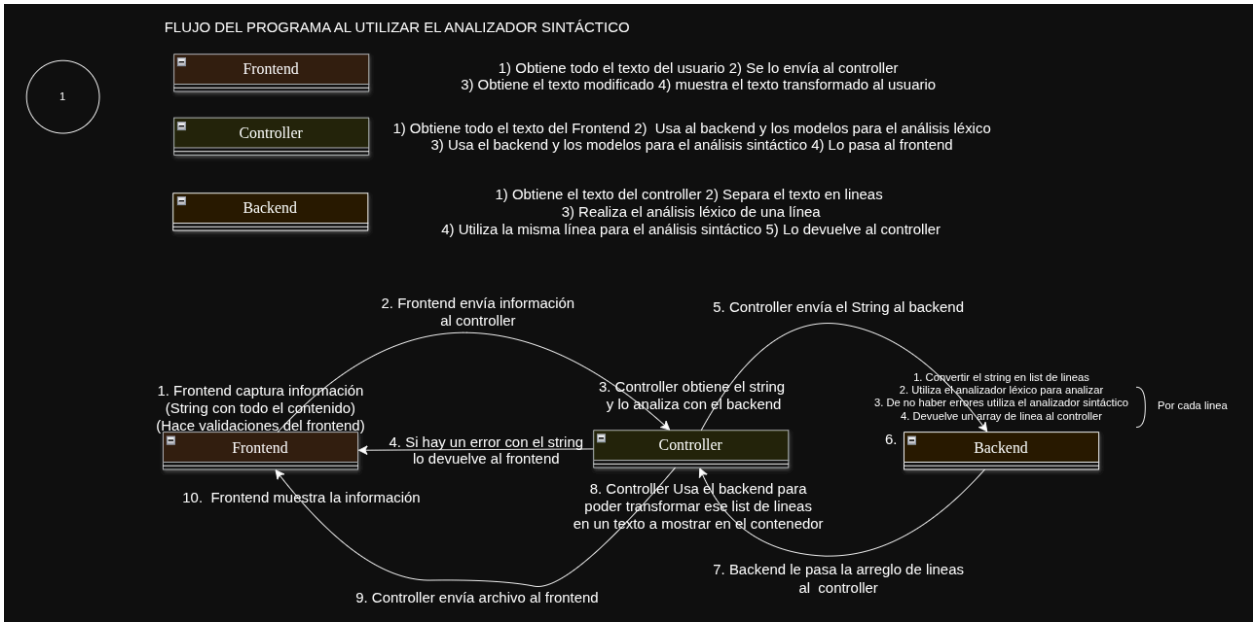
Para representar gráficamente el programa se plantea un diseño de distintas partes del programa.

Fase de Diseño:

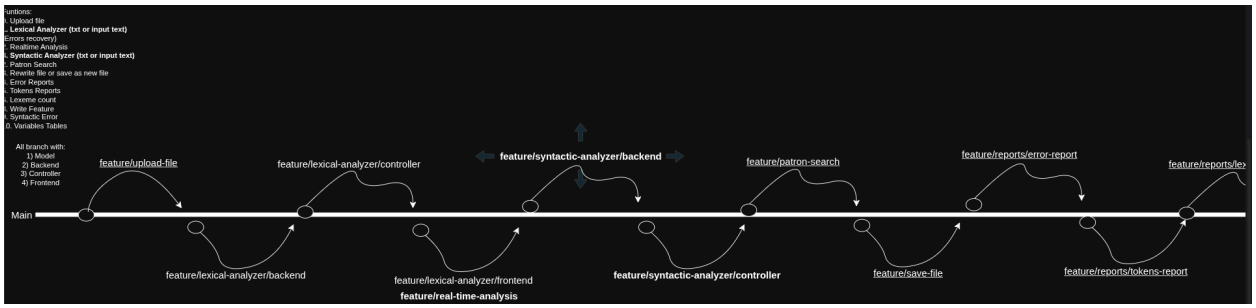
1. Bosquejo de la apariencia del programa



2. Modelo del flujo del programa



3. Uso de las ramas en git

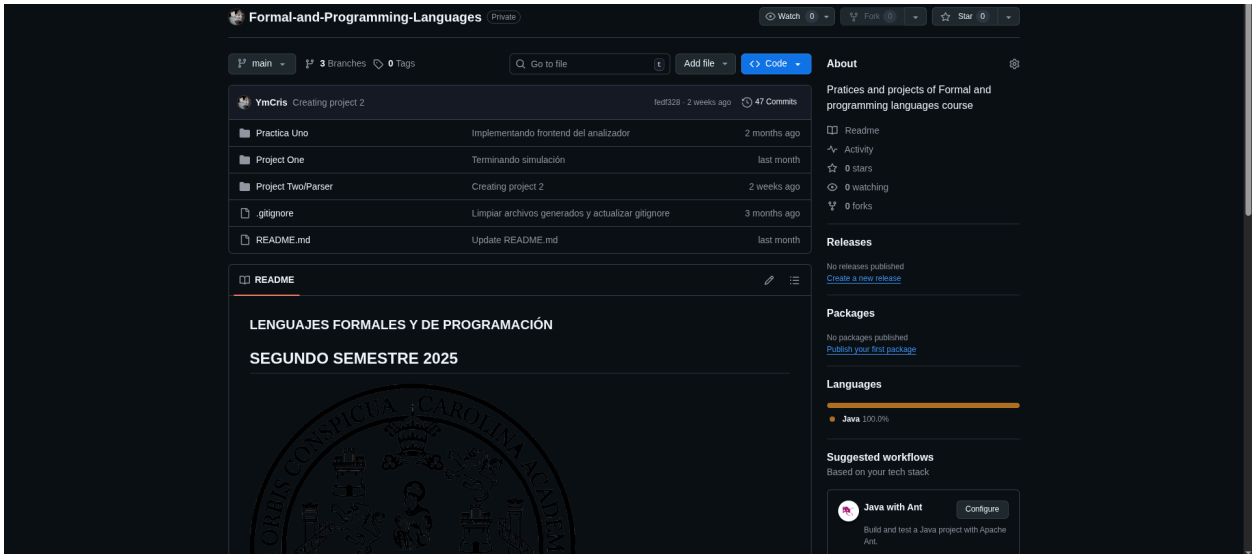


Algunas ramas fueron realizadas en base a “ramas hijas”, y diseñadas según su función usando prefijos como “feature” para funcionalidades, “fix” para arreglo de bug, etc.

· Repositorio en GitHub:

Para una mejor descripción de los avances en el transcurso de la creación del programa se describen detalladamente los cambios en los commits del repositorio así como el de sus ramas.

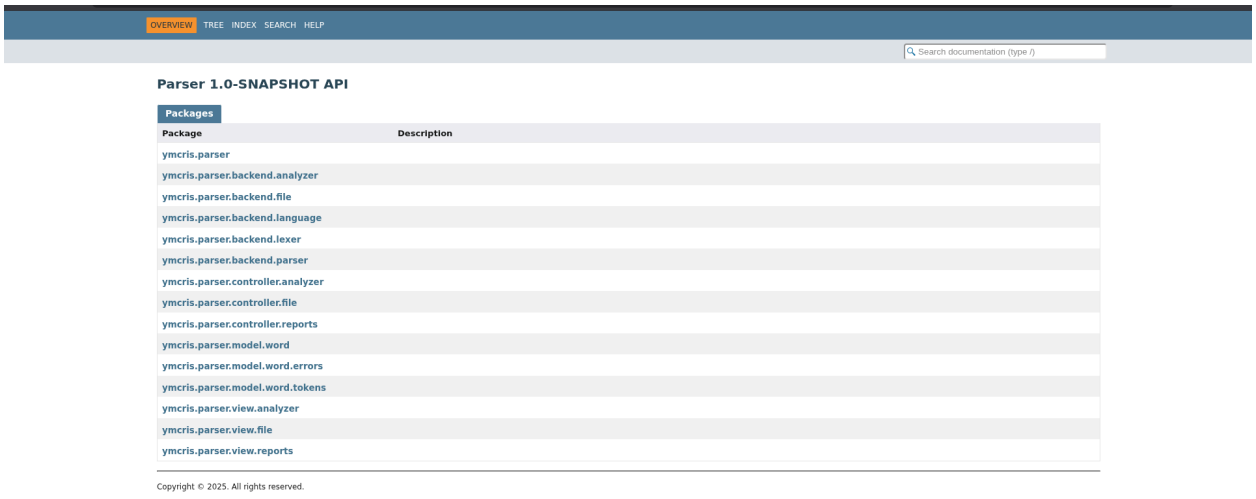
Repositorio en GitHub:



· Documentación en JavaDoc:

Para representar los comentarios en código se incluye dentro de la carpeta target el index del Overview.

Overview (SyntacticAnalyzer 1.0-SNAPSHOT API)



Problemas del Programa

En la compañía, entendemos que la satisfacción del cliente es nuestra prioridad número uno. Por eso, nos comprometemos a ofrecer un servicio de calidad que garantice una experiencia sin problemas al utilizar nuestros productos. En caso de que encuentres algún inconveniente durante la descarga, instalación o ejecución de nuestro programa, estamos aquí para ayudarte. Nos comprometemos a brindar asistencia técnica y, si es necesario, realizar devoluciones dentro de un plazo de 12 horas después de tu compra.

Es importante destacar que la mayoría de los problemas que puedas encontrar suelen ser el resultado de una ejecución incorrecta, pero no te preocupes, estamos aquí para guiarte en cada paso del proceso. Simplemente reiniciando el programa o volviendo a instalarlo, muchos de estos problemas se pueden resolver fácilmente.

Sin embargo, debemos señalar que cualquier modificación no autorizada al programa o descarga incorrecta podría limitar nuestra capacidad para abordar el problema. En tales casos, nos reservamos el derecho de no proporcionar asistencia.

Nuestro equipo de soporte técnico está siempre disponible para ayudarte a resolver cualquier inconveniente que puedas encontrar. Queremos que tengas la mejor experiencia posible con nuestros productos y haremos todo lo posible para asegurarnos de que así sea.